

INTERNSHIP PROJECT REPORT

# Diya

An unattended meditation kiosk

---

PRESENTED BY

**Ayush Gupta**

UNDER THE GUIDANCE OF

**Eshwar Teja**

Mentor



## INTRODUCTION

# What I worked on

A visitor walks up to a kiosk with nobody there to help them. My work was everything around the experience: identifying the visitor, running their session, and handing back a report — then resetting for the next person.

01

### The kiosk application

A fullscreen desktop app in C# and .NET with Avalonia, running on Linux.

02

### Phone-based identification

A backend and web pages so a visitor is identified using only their own phone.

03

### Hardware integration

Connecting to the other team's application without touching their code.

04

### Packaging and deployment

A single installable package that boots straight into the kiosk and restarts itself.

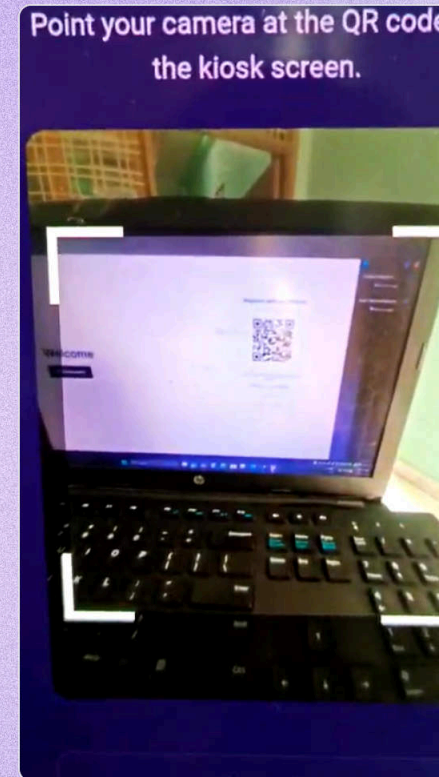


#### THE QR CODE SYSTEM

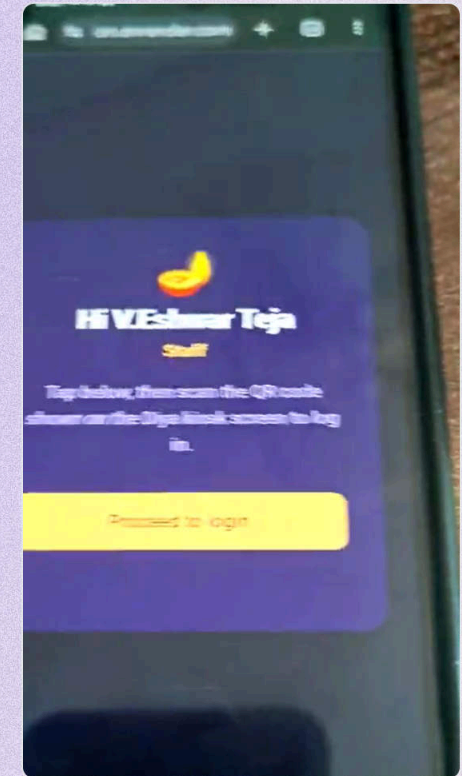
# The kiosk shows the code. The phone reads it.

There was no scanner hardware, so I inverted it. The kiosk asks the backend for a session and displays it as a QR code. The visitor's phone scans that code, and the kiosk — which is polling the backend — moves ahead on its own.

- No scanner, no app install, no typing
- The phone is the camera and the keyboard
- Phone and kiosk never talk directly — only through the backend



The scan itself, on the real setup

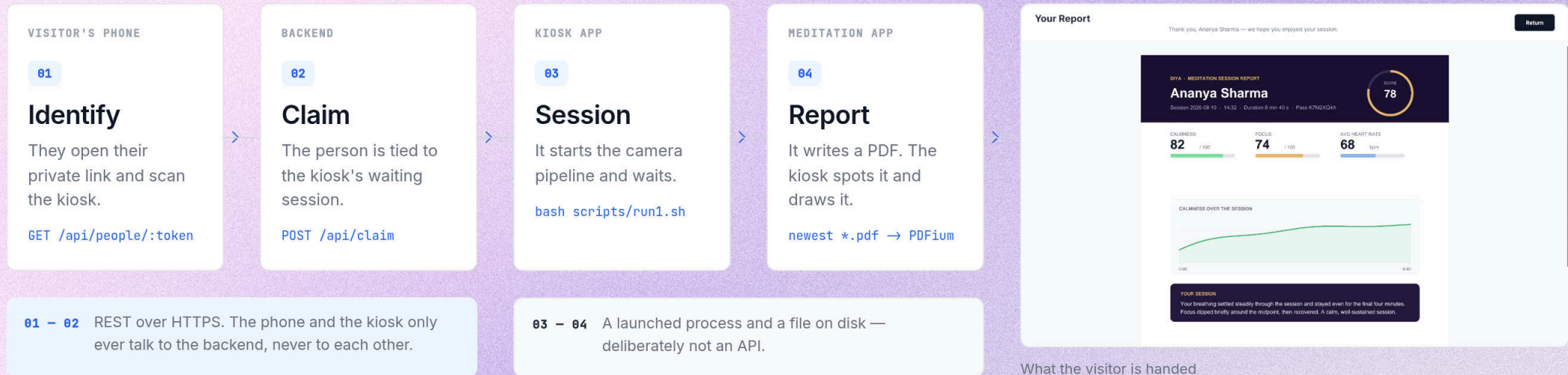


The site greets them, then opens the camera



END TO END

# The whole flow



What the visitor is handed

Nobody presses a button to begin. Being identified is what starts the session.



# Two real problems

## Problem 01

### Identifying someone with no scanner

Every obvious answer needed hardware — a QR reader, a card reader, or a keyboard strangers would share. I tried putting the visitor's data in the QR code, then putting an ID in it, and both still needed a reader.

#### APPROACH

Turn it around: the kiosk displays the QR and the visitor's phone does the scanning. The constraint made the design simpler.

## Problem 02

### Talking to another team's app without changing it

The meditation software shipped as a separate .deb application. It had no API to call, and I could not modify it — it kept changing on their side all summer.

#### APPROACH

My app launches their program and then simply watches the folder. It keeps checking whether the report PDF has been created, and only accepts a file newer than the moment the session started — so nobody gets the previous visitor's report. Once the file appears, the kiosk renders it.

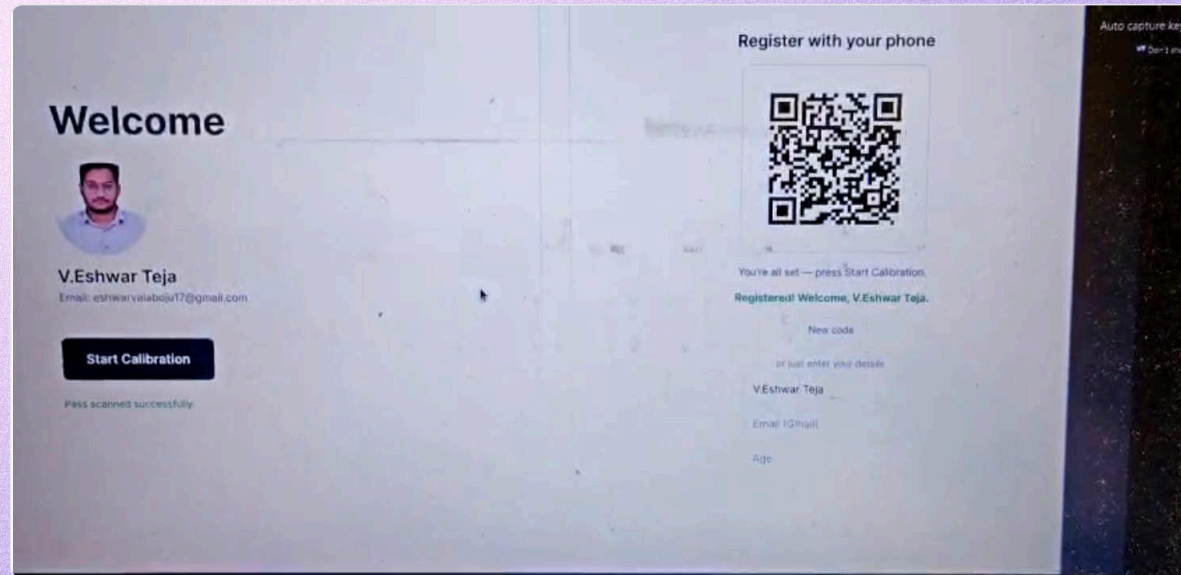
One process to launch and one file to watch — so their internals could change freely without ever breaking my application.



SEE IT RUN

# The full walkthrough

Recorded on the real setup: the phone logs in, scans the kiosk, and the session begins on its own.



▶ Play the video



THANK YOU

# Thanks for the opportunity

This internship taught me a great deal. I got to work in C# and .NET, build a real desktop application with Avalonia, write a backend and web pages, package software for Linux, and integrate with hardware I did not control. All of it will help me going forward.

**Eshwar Teja**

For mentoring me through the project.

**Kalyan Sir**

For his encouragement throughout.